



Data Structures
CS 246 - 001
Department of Physics and Computer Science
Medgar Evers College
Exam 3

Instructions:

- The exam requires completing a set of tasks within 80 minutes.
- Write all solutions in the blue book provided.
- The list of data structures is provided.
- For any coding problem, assume you have access to the libraries `'iostream'`, `'string'`, `'sstream'`, `'cctype'`, `'cstdlib'`, `'ctime'`, and `'stdexcept'`.
- Notes are not allowed.
- Cheating of any kind is prohibited and will not be tolerated.
- **Violating and failing to follow rules will result in an automatic zero (0) for the exam.**

TO ACKNOWLEDGE THAT YOU HAVE READ AND UNDERSTOOD THE INSTRUCTIONS ABOVE,
PRINT YOUR NAME AND THE DATE ON BOTH THIS SHEET AND THE BLUE BOOK

Name: _____

Date: _____

Grading

Problem	Maximum Points	Points Earned
01	2.0	
02	3.0	
03	2.0	
04	2.5	
05	2.5	
06	2.5	
07	3.0	
08	1.5	
09	1.0	
Total	20.0	

- Construct the runtime table and calculate the runtime function of the following function for its worst-case scenario. You must initially state the runtime factor (input size) n , and let 1 be the cost of every operation.

```

01.  template <typename T>
02.  void C(Array<T>& data, size_t idx)
03.  {
04.      if(idx < data.size())
05.      {
06.          T k = data[idx];
07.          size_t i = idx;
08.
09.          while(i > 0 && data[i-1] > k)
10.          {
11.              data[i] = data[i-1];
12.              i -= 1;
13.          }
14.          data[i] = k;
15.      }
16.  }

```

- Write a void function named SelectionSort() whose header is

```

template <typename T>
void SelectionSort(Node<T>* ptr)

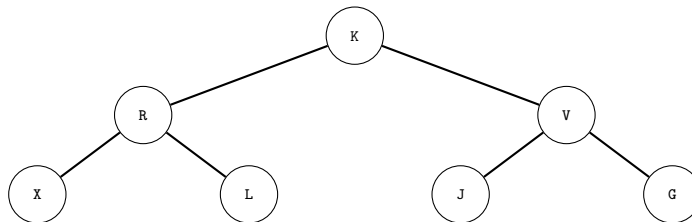
```

and definition is the linked list version of the SelectionSort() algorithm. Remember, a linked list can be empty.

- Use the master theorem to determine the theta notation of the following recurrence functions. You must show work [proof of the case] to receive full credit.

- $T(n) = 5T(n/5) + 10$
- $T(n) = T(2n/3) + 3n$
- $T(n) = 16T(n/4) + n\sqrt{n}$
- $T(n) = 2T(n/16) + \sqrt[4]{n}$

- Write each tree traversal [infix, prefix, or postfix] of the following binary tree. The values must be written in a line, and each traversal must be preceded by its name.



- A heterogram is a word with no repeating letters. Write the definition of the Boolean function

```

bool isHeterogram(string str)

```

that returns true if *str* is a heterogram; otherwise, it returns false. Your function must use at least one of the data structures provided. Furthermore, a word consists only of letters.

6. The previous problem can be implemented more efficiently with hashing. Prove it by
- stating the data type of the key of the hash function.
 - stating the number of slots (elements) of the hash table.
 - defining the hash function.
7. A stack can be implemented with two queues. Given that the generic class *ImitateStack* has a private generic *Queue* array field named *data* with a size of 2, define
- its `push()` method
 - its `pop()` method

Precede each method header with a template statement.

8. Write a Boolean function named `isLeaf()` whose header is

```
template <typename T>
bool isLeaf(Tnode<T>* ptr)
```

and returns true only if *ptr* is a leaf of a linked binary tree; otherwise, it returns false. Remember, a linked binary tree can be empty.

9. Define each of the following special binary trees:
- Full Binary Tree
 - Max-Heap

Data Structures Reference

Array	
operator[] (idx)	returns the item at index <i>idx</i> in the array.
size()	returns the size of the array.
length()	returns the size of the array.
ArrayList	
append(itm)	adds <i>itm</i> to the end of the arraylist.
insert(idx,itm)	inserts <i>itm</i> at index <i>idx</i> in the arraylist.
remove(idx)	removes the item at index <i>idx</i> from the arraylist.
detach()	removes the last item from the arraylist.
find(itm)	returns the index of the first occurrence of <i>itm</i> in the arraylist.
contains(itm)	checks if <i>itm</i> exists in the arraylist.
at(idx)	returns the item at index <i>idx</i> in the arraylist. (equivalent to operator[])
empty()	checks if the arraylist is empty.
length()	returns the size of the arraylist.
Node	
data()	returns the data of the node.
prev()	returns the previous link of the node.
next()	returns the next link of the node.
Pair	
key()	returns the key of the pair.
value()	returns the value of the pair.
Map	
put(key,value)	inserts (<i>key</i> , <i>value</i>) pair into the map.
get(key)	returns value associated with <i>key</i> in the map. (equivalent to operator[])
remove(key)	removes the pair associated with <i>idx</i> from the map.
contains(key)	checks if <i>key</i> exists in the map.
empty()	checks if the map is empty.
size()	returns the size of the map.
Set	
insert()	adds the object to the set if it is not already present.
remove()	removes the object from the set.
contains()	returns true if the object exists in the set; false otherwise.
empty()	returns true if the set contains no objects.
size()	returns the total number of objects in the set.
Multiset	
insert()	adds the object to the multiset if it is not already present.
remove()	decrements the count of the object by one.
removeAll()	removes all instances of the object from the multiset.
contains()	returns true if the object exists in the multiset; false otherwise.
count()	returns the number of times the object appears in the multiset.
empty()	returns true if the multiset contains no objects.
size()	returns the total number of objects across all entries.
Stack	
push()	adds an item to the top of the stack.
pop()	removes the top item from the stack.
top()	returns the top item of the stack.
empty()	checks if the stack is empty.
Queue	
enqueue()	adds an item to the back of the queue.
dequeue()	removes the front item from the queue.
peek()	returns the front item of the queue.
empty()	checks if the queue is empty.
Deque	
insertFirst(obj)	adds an item to the front of the deque.
insertLast(obj)	adds an item to the back of the deque.
removeFirst()	removes the front item from the deque.
removeLast()	removes the back item from the deque.
first()	returns the front item of the deque.
last()	returns the back item of the deque.
empty()	checks if the deque is empty.
Tnode	
data()	returns the data of the tree node.
left()	returns the left link of the tree node.
right()	returns the right link of the tree node.
p()	returns the parent link of the tree node.